

The Determinism Contract: Full Operational Treatment of Substrate Determinism, Model Non-Determinism, and the Five Guarantees That Distinguish Them in CKS

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 5, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, at full operational depth, the structure of the determinism contract introduced in the parent foundational note A1.10 — naming the substrate-versus-model determinism boundary, the five guarantees that compose into the substrate determinism commitment, the five categories of allowed non-determinism, regression testing as the contract's operational application, and the contract's load-bearing role for downstream architectural commitments — so that the subsequent decomposition notes A2.56 through A2.63 can each specialize one component of the contract as a standalone derivation.

Abstract

The CKS determinism contract introduced in A1.10 collected the source paper's distributed determinism commitments under a single contract name, citing §4.1 (the substrate-versus-model determinism boundary at the governance frontier), §6.2 (the substrate determinism that distinguishes external structured memory from per-session in-context memory), §11.3 (the substrate as the source of truth for what was decided, by whom, under what authority, with what rationale), and §4.2 (the LLM as substrate mediator with non-determinism bounded to the non-substrate layer). A1.10 is the synthesis. This note is the integrating frame for the contract's operational decomposition: it specifies the structure of the contract — the load-bearing substrate-versus-model distinction in §2, the five guarantees in §3, the allowed non-determinism in §4, regression testing in §5, the limits of the contract's scope in §6, and an integrating-frame operational test in §7 — without specializing any component. The specializations are reserved for A2.56 (representation versus model-output determinism), A2.57 through A2.61 (the five guarantees), A2.62 (the five allowed non-determinism categories), and A2.63 (regression testing). The contribution of this note is the integrating structure that organizes the decomposition.

1. Why the contract needs a full operational treatment as standalone

The parent foundational note A1.10 names the determinism contract as a synthesis of commitments distributed across the source paper. That synthesis is sufficient to reference the contract by a single name. It is not sufficient to *operationalize* the contract — to specify which determinism is required

where, which is allowed to vary, how the required determinism decomposes into testable guarantees, and how the contract's components compose with the architecture's other commitments. The operational specification is what implementations and audits need; the synthesis is what citations need. The two roles are distinct, and the operational specification is large enough that it does not fit inside the synthesis without diluting either it or the foundational note.

The motivating cases are deployments where the determinism properties of substrate behavior must be operationally specified rather than referenced as a single named commitment. Such deployments must distinguish the determinism the architecture requires (substrate representation determinism) from the determinism it does not require (LLM model-output determinism), and they must distinguish the substrate-internal scope of the contract from any commitment the architecture might be misread as making about external systems or about underlying infrastructure. Without that operational specification, implementations consistently drift toward one of two failure modes. They over-engineer for full determinism — requiring deterministic LLM outputs, which is operationally infeasible for current LLM technology and forecloses the AI-as-substrate-mediator role per A1.04. Or they under-engineer — allowing non-determinism throughout the architecture, which breaks path retraceability per A1.07 and source-of-truth per A1.08. The integrating-frame treatment is what specifies the boundary between the two failure modes precisely enough for the boundary to be implemented and verified.

A second motivation is the strategic prior-art posture of the derivation series. The determinism contract is consequential prior art because it forecloses architectures that conflate substrate determinism with LLM determinism in either direction. Patentable derivations that focus on deterministic AI architectures, replayable coordination systems, or audit-coherent AI infrastructure are substantially more defensibly contested when the contract is publicly formalized as standalone with the substrate-versus-model boundary specified precisely and the five guarantees enumerated as the components a downstream system must satisfy.

A third motivation is the connection to A1.04 (AI as substrate mediator). A1.04 commits to the LLM operating as substrate mediator with non-determinism bounded to the non-substrate layer. The determinism contract is what specifies this bounding architecturally: the substrate layer is deterministic per the five guarantees; the LLM layer is allowed non-determinism per A2.62's five categories. The two commitments compose. A1.04's Property A (LLM reads from substrate as primary source) and Property B (LLM writes under orchestration rules), as decomposed in A2.19 and A2.20, name the read and write directions of the mediator role; the determinism contract names the determinism properties the mediator role's reads and writes must respect. Property C and Property D bound LLM non-determinism to the non-substrate layer; the contract specifies what that bounding requires of the substrate layer.

2. The substrate-versus-model determinism distinction

The contract operates on a distinction between two determinism properties that the architecture treats asymmetrically. A2.56 formalizes the distinction standalone; this note names it at the integrating level.

Substrate (representation) determinism. The substrate's representation, persistence, and read behavior compose deterministically. Two reads of identical substrate state yield identical content; two cell executions under the same orchestration rules over the same substrate state produce equivalent substrate writes; every change to substrate state is addressable through the substrate's standard read operations as new substrate state, not as an unrecorded side effect. This is the determinism the contract requires. It is a property of the substrate layer, not of any computation that operates over the substrate.

Model-output (LLM) determinism. LLM outputs vary across executions, even with identical inputs, due to sampling, temperature, and other model-internal non-determinism. This is the determinism the contract does not require. Requiring it would be operationally infeasible for current LLM technology and would foreclose the AI-as-substrate-mediator role: a deterministic-output LLM is not what the architecture needs, and the variability of LLM outputs is bounded to the non-substrate layer rather than eliminated.

The distinction is load-bearing because the architecture's downstream commitments — source-of-truth per A1.08, path retraceability per A1.07, AI-as-substrate-mediator per A1.04 — each depend on the substrate side of the distinction and not on the model side. Misreading the scope in either direction breaks the contract. Binding LLM outputs makes the contract impossible to satisfy. Binding nothing in particular makes the substrate's commitments evaporate. The contract binds the representation; the model is out of scope.

3. The five guarantees that compose into substrate determinism

The substrate determinism commitment decomposes into five guarantees, each operating at a distinct level of substrate behavior. The five are not arbitrary; each addresses a specific aspect of substrate behavior that a downstream commitment depends on. A2.57 through A2.61 formalize each guarantee standalone.

Guarantee A — Same substrate state yields the same content. For any read against substrate, the same substrate state produces the same content. This guarantee operates at the read level. It is what makes the substrate inspectable per §2.1 of the source paper and what makes it usable as the source-of-truth's authoritative answer to *what is currently the case*. A2.57 formalizes Guarantee A standalone.

Guarantee B — Same substrate state plus same orchestration rules yields equivalent cell behavior. For any cell execution under specified orchestration rules, the same substrate state and the same rules produce equivalent cell behavior. *Equivalent* is operationally specified in A2.58: it does not require bit-identical cell behavior, which would be incompatible with the LLM non-determinism bounded to the non-substrate layer per A1.04. It requires behavior that satisfies the rule's specification at the substrate-write layer — the same substrate writes, the same state changes, the same orchestration rule selected — with non-determinism bounded to the LLM's intermediate outputs that do not propagate to substrate. A2.58 formalizes Guarantee B standalone.

Guarantee C — Substrate changes are addressable. Every change to substrate is itself substrate content, with provenance per A2.40 and addressability through the substrate's standard read operations

per A2.25. Changes do not occur as side effects that exist outside substrate. This guarantee operates at the change level and is the property A1.07's path retraceability commitment depends on at the contract layer. A2.59 formalizes Guarantee C standalone.

Guarantee D — Conflict states are preserved. Conflict states per A1.03 (conflict as first-class substrate state) are preserved across reads and operations. Resolution does not erase the conflict; it is recorded as a resolution decision under A2.13's preservation commitment, and the contradiction it resolved remains addressable per Guarantee C. This guarantee operates at the conflict level and is what prevents non-deterministic processes from silently selecting between contradictions on the substrate's behalf. A2.60 formalizes Guarantee D standalone.

Guarantee E — Substrate is the source of truth. Substrate is authoritative for the five categories of state per A1.08 and A2.42 through A2.47. External systems do not override the substrate's determinism properties; the contract is a substrate-internal commitment, not a system-wide commitment shared with whatever external systems the substrate composes with. A2.61 formalizes Guarantee E standalone.

4. Allowed non-determinism — five categories

The contract permits non-determinism in five specific categories. A2.62 formalizes the categories standalone; this note names them at the integrating level.

(a) LLM token output. LLM token outputs vary across executions due to sampling, temperature, and other model-internal non-determinism. The variation is bounded to LLM-internal output and does not propagate to substrate, because LLM writes occur under orchestration rules with rule-governed structure per Property B (A2.20).

(b) Cell execution timing. Cells may execute at varying times and in varying orders. Substrate content depends on the order in which writes are committed per A2.10, but the timing of cell execution is not determined by the contract.

(c) Inter-cell communication. Cells communicate through substrate per A2.12 (no direct cell-to-cell channels), but the timing at which one cell's writes become visible to another cell is not determined by the contract.

(d) External-system responses. Adjacent components per A1.16 (hybrid systems) may respond non-deterministically; the contract commits to substrate determinism, not to external-system determinism.

(e) Hardware and infrastructure. Underlying hardware, networking, and storage may produce non-deterministic effects (latency variation, transient failures, resource contention). The architecture is technology-agnostic per A1.05 and does not commit to specific infrastructure determinism.

5. Regression testing as operational application

The contract is operationally testable through regression testing. A regression test fixes substrate state and orchestration rules, executes specified operations, and verifies that the five guarantees hold. The

contract becomes operationally enforceable through the test infrastructure; without regression testing, the contract would be aspirational. A2.63 formalizes regression testing standalone, including the test structure for each of the five guarantees and the relationship between regression-test coverage and the contract's coherence as a substrate property over time.

6. What the contract does not claim

The contract is precise about what it commits to, and equally precise about what it does not commit to. The negative scope below is named at the integrating level so that the contract is not over-read by downstream consumers.

It does not claim LLM determinism. LLM outputs are non-deterministic per category (a) of §4 above; the architecture does not require LLM determinism and is operationally compatible with current LLM technology.

It does not claim infrastructure determinism. Hardware, networking, and storage non-determinism per category (e) is allowed; the architecture is technology-agnostic per A1.05.

It does not require bit-identical cell behavior under Guarantee B. *Equivalent cell behavior* is rule-specified at the substrate-write layer and may include allowed non-determinism in non-substrate outputs. What is required is rule-conformance, not bit-identity.

It does not specify implementation patterns for determinism. Implementations may use various mechanisms — transactional storage, immutable-data architectures, version control, content-addressed storage, append-only logs — to satisfy the five guarantees. The architectural commitment is to the guarantees being operationally satisfied; specific implementation patterns are deployment choices.

It does not commit to determinism across substrate migrations beyond migration safety. A migrated substrate satisfies the five guarantees on its new host; cross-host determinism beyond what migration safety per A1.05 specifies is not architecturally committed.

It does not claim that determinism is observable without regression testing. Operational verification of the contract requires the test infrastructure per A2.63; determinism is operationally testable, not observationally automatic.

7. Operational test at the integrating-frame level

A system instantiates the determinism contract at the integrated level if and only if all of the following are true at all times during the substrate's existence.

1. **Distinguishes substrate from model-output determinism per A2.56.** Substrate is required to be deterministic in the representation sense; LLM outputs are not required to be deterministic. The architecture treats the two layers asymmetrically.
2. **Satisfies all five guarantees per A2.57 through A2.61.** Same substrate state yields the same content (Guarantee A); same state plus same rules yields equivalent cell behavior (Guarantee B); substrate changes are addressable (Guarantee C); conflict states are preserved (Guarantee D);

substrate is source of truth (Guarantee E).

3. Bounds non-determinism to the five allowed categories per A2.62. Non-determinism in LLM token output, cell execution timing, inter-cell communication timing, external-system responses, and hardware and infrastructure does not propagate from the LLM layer or external systems into substrate representation.

4. Supports regression testing per A2.63 as the operational verification of the contract. The test infrastructure verifies the five guarantees over time, under change in substrate, orchestration rules, the LLM, or the deployment environment.

5. Operates substrate-internally. External systems do not override the contract; the contract's scope is the substrate layer and the architecture's commitments at that layer per Guarantee E.

A system that fails any of (1) through (5) does not instantiate the contract at the integrated level. The individual operational tests for the substrate-versus-model distinction, each of the five guarantees, the allowed non-determinism categories, and regression testing are specified in A2.56 through A2.63.

8. Why naming the integrating frame as standalone matters

Implementations of AI-coupled coordination systems consistently drift toward either over-claiming determinism (requiring deterministic LLM outputs, which fails operationally) or under-claiming determinism (allowing non-determinism throughout, which breaks retraceability and source-of-truth). The drift is steady because the substrate-versus-model boundary is operationally subtle: substrate and LLM outputs interact at cell-execution boundaries per A2.10, and the contract specifies precisely which side of the boundary which determinism applies to. Without an integrating-frame treatment naming the boundary, the five guarantees, the allowed non-determinism, and regression testing as a single named structure, implementations under deployment pressure consistently make local decisions that violate one component of the contract while preserving others — producing systems where determinism properties are ambiguous, where substrate may be deterministic in some operational paths but not others, where LLM outputs are incorrectly required to be deterministic, where conflict states are erased by non-deterministic resolution, or where substrate writes are non-addressable.

The downstream consequences manifest as path-retraceability failures (Guarantee C violations), source-of-truth fragmentation (Guarantee E violations), conflict-handling inconsistencies (Guarantee D violations), and cell-execution divergence under identical rules (Guarantee B violations). Each of these is an architectural failure rather than a bug, and each is invisible at the level of any single transaction; the failures show up only when the system is exercised across enough operational paths to expose the contract violation. The integrating-frame treatment is what makes those failures namable in advance — and therefore designable around — rather than discoverable only in production.

Naming the determinism contract integrating frame as a standalone architectural commitment, with the substrate-versus-model boundary in §2, the five guarantees in §3, the allowed non-determinism in §4, regression testing in §5, the negative scope in §6, and the integrating-frame operational test in §7, gives downstream readers a precise specification of what the determinism commitment is. A2.56 through A2.63 specialize each component; together with this integrating frame, they give the full operational

decomposition of A1.10 that downstream implementers, auditors, and critics can use to argue with the contract in shared terms.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Determinism Contract: Full Operational Treatment of Substrate Determinism, Model Non-Determinism, and the Five Guarantees That Distinguish Them in CKS*. May 5, 2026. ORCID: 0009-0004-8065-3235.